

Vector Quantized Models for Planning

-- ICML2021

Hongming Zhang
2021.07.14

Contents



- Introduction
- Model-based Planning Formulations
- VQHybrid
- Experiments

Introduction



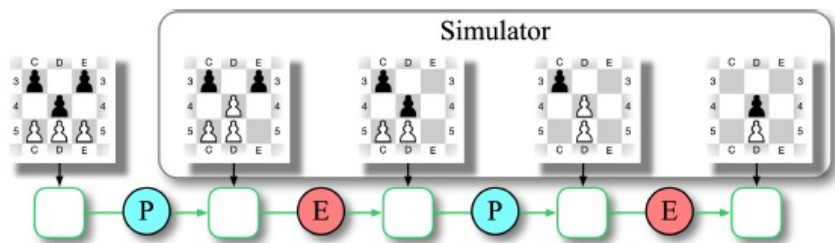
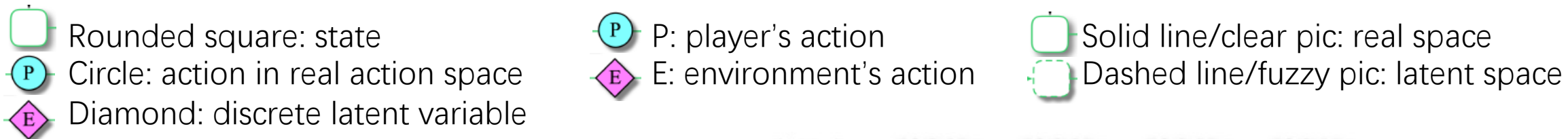
- Goal: handle **stochastic** and partially-observable environments in planning
- Insight: use discrete **autoencoders** to capture the multiple possible effects of an action in a stochastic environment

Contents

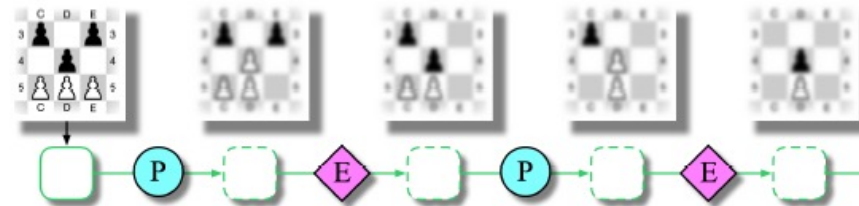


- Introduction
- Model-based Planning Formulations
- VQHybrid
- Experiments

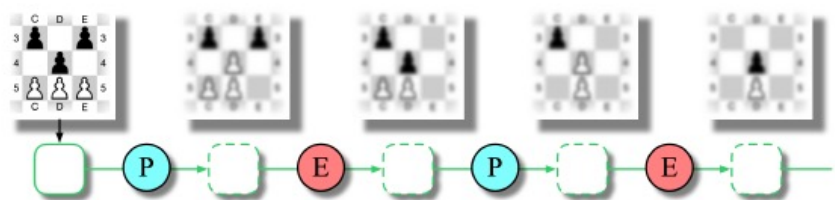
Model-based Planning Formulations



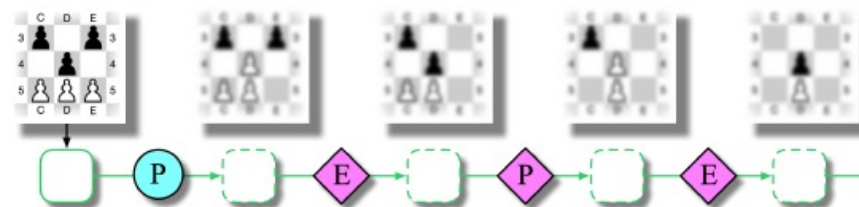
(a) AlphaZero



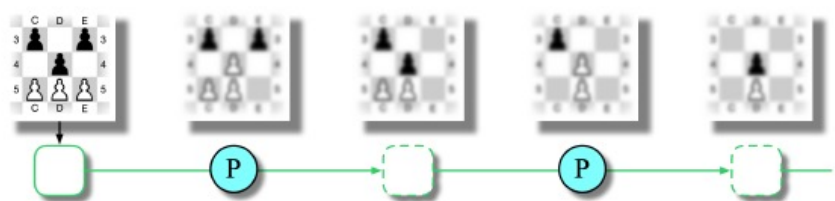
(d) VQHybrid



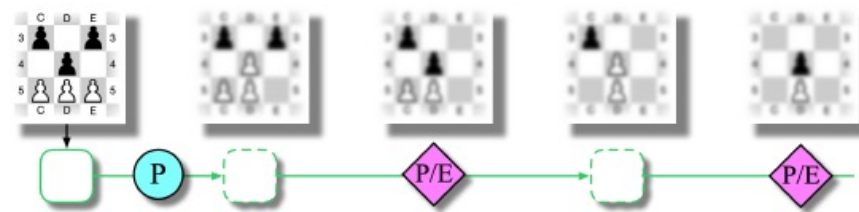
(b) Two-player MuZero



(e) VQPure



(c) Single-player MuZero



(f) VQJumpy

Model-based Planning Formulations

Table 1. Key features of different planning algorithms.

Method	Learned Model	Agent Perspective	Stochastic	Abstract Actions	Temporal Abstraction
AlphaZero	✗	✗	✗	✗	✗
Two-player MuZero	✓	✗	✗	✗	✗
Single-player MuZero	✓	✓	✗	✗	✗
VQHybrid	✓	✓	✓	✗	✗
VQPure	✓	✓	✓	✓	✗
VQJumpy	✓	✓	✓	✓	✓

Contents



- Introduction
- Model-based Planning Formulations
- VQHybrid
- Experiments

VQHybrid



- Techniques:

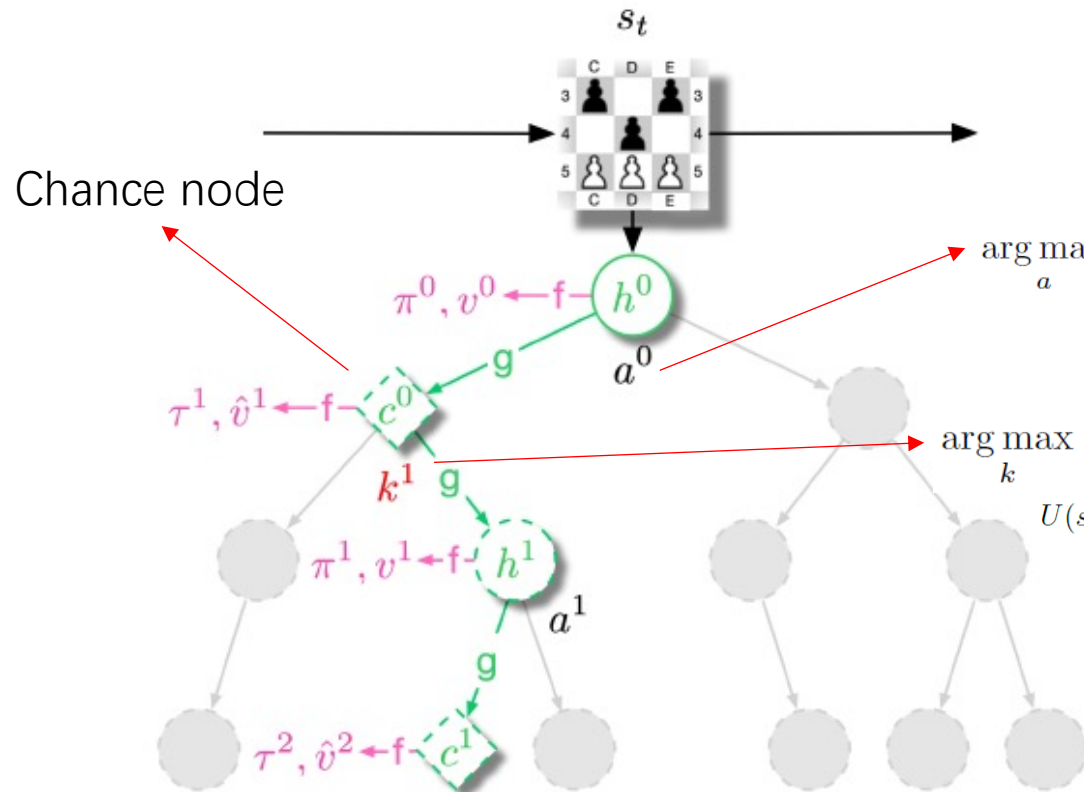
Monte Carlo Tree Search (MCTS) + Vector-Quantized Variational AutoEncoders (VQVAE)

- Instances:

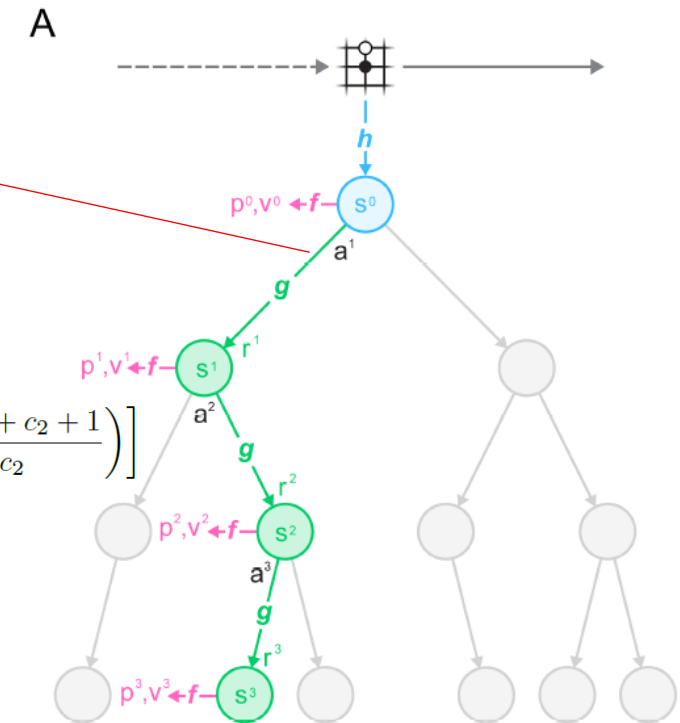
MuZero + State VQVAE

Planning Encode a sequence of states and actions into a sequence of discrete latent variables

VQHybrid - Planning

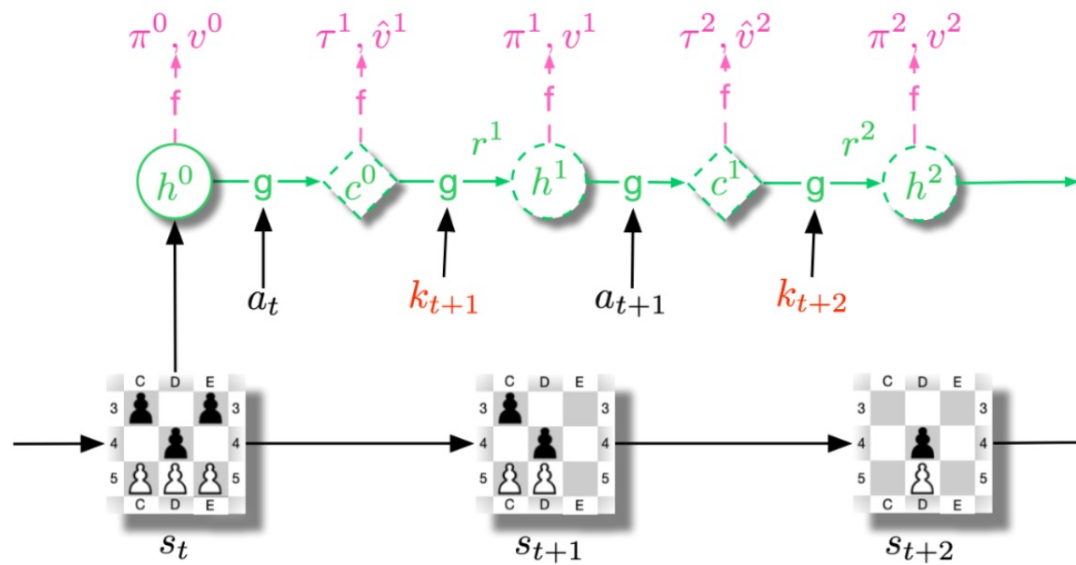


VQHybrid



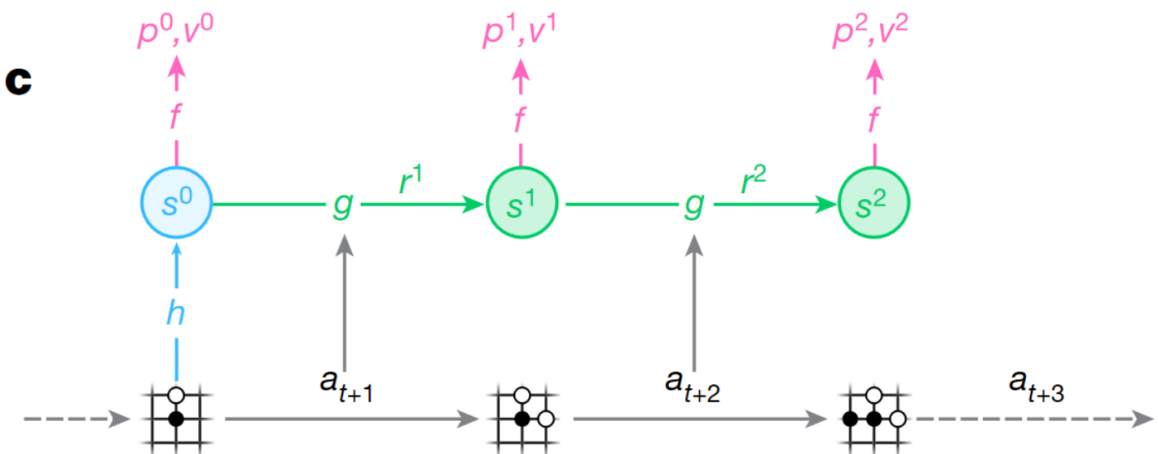
MuZero

VQHybrid - Planning



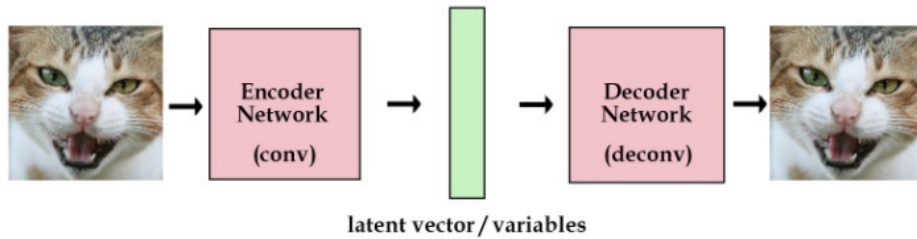
VQHybrid

c

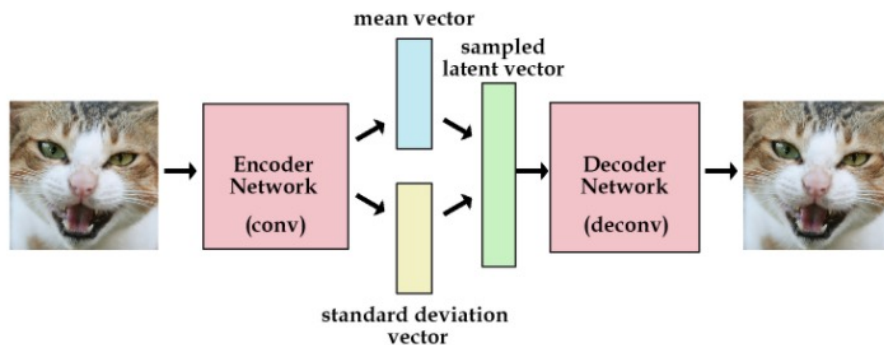


MuZero

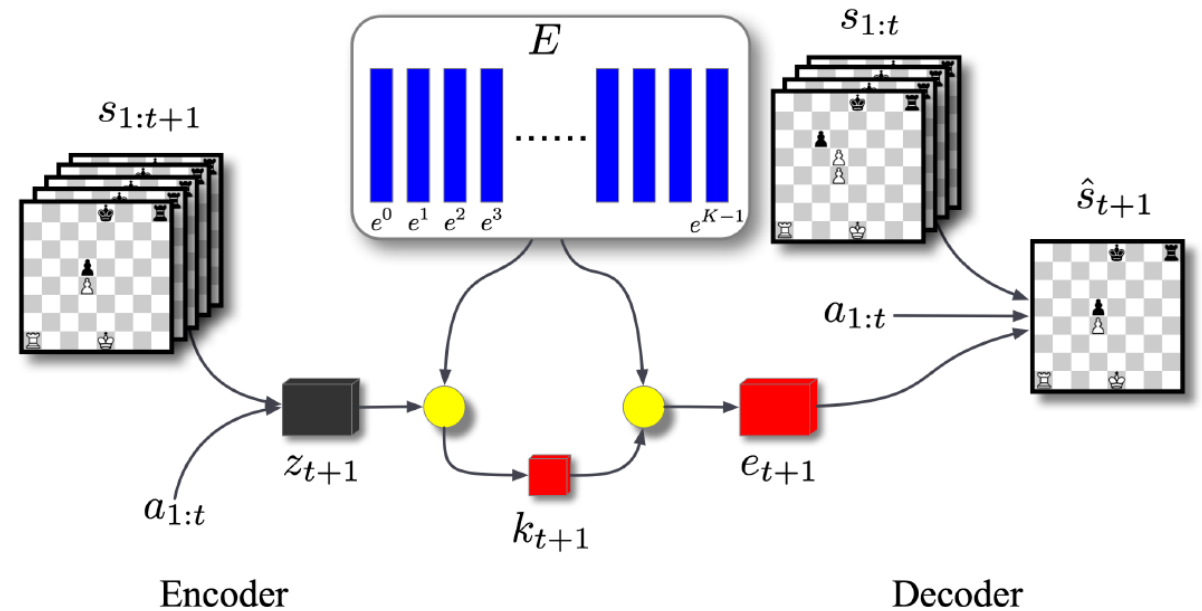
VQHybrid - AutoEncoder



AutoEncoder (AE)

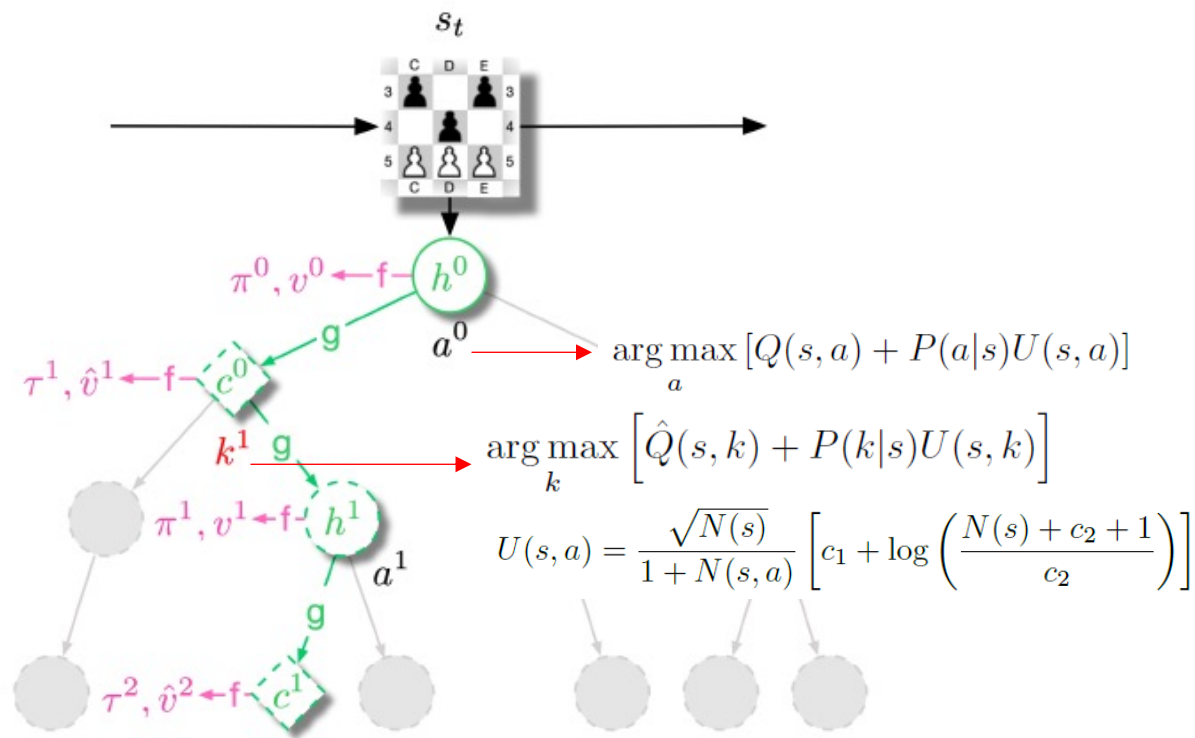
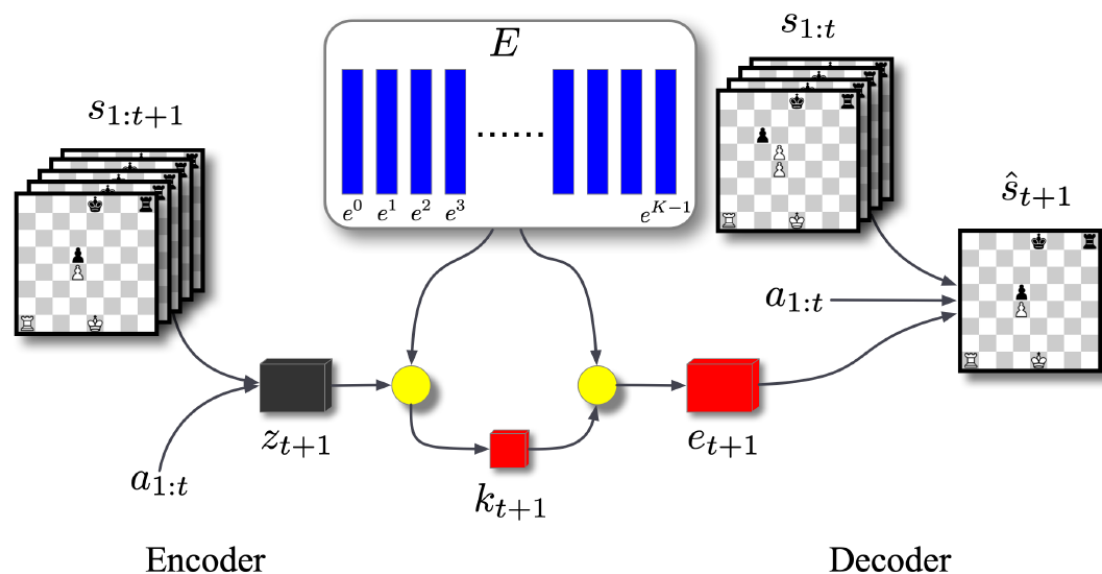


Variational AutoEncoder (VAE)



State Vector Quantised-Variational AutoEncoder (State VQVAE)

VQHybrid - AutoEncoder



VQHybrid - Training



Two-stage training process

1. State VQVAE

Train the state VQVAE model which encodes the observations into discrete latent variables

2. Transition model (MuZero)

Train the transition model using the discrete latent variables learned by the state VQVAE model

VQHybrid - Training

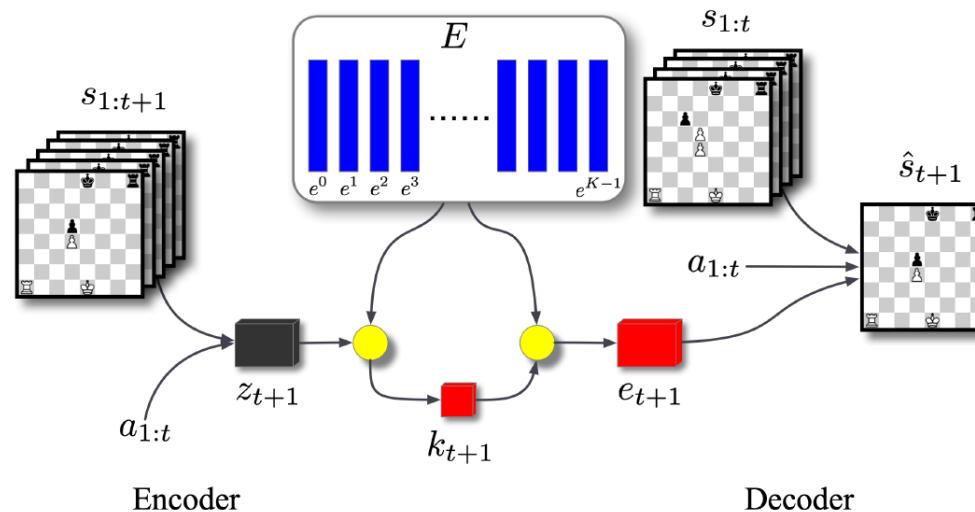
Two-stage training process

1. State VQVAE

Train the state VQVAE model which encodes the observations into discrete latent variables

2. Transition model (MuZero)

Train the transition model using the discrete latent variables learned by the state VQVAE model



VQHybrid - Training



Two-stage training process

1. State VQVAE

Train the state VQVAE model which encodes the observations into discrete latent variables

2. Transition model (MuZero)

Train the transition model using the discrete latent variables learned by the state VQVAE model

$$\frac{1}{M} \sum_{m=0}^{M-1} \text{CE}(a^m, \pi(h^{2m})) + \frac{1}{M} \sum_{m=1}^M \text{CE}(k^m, \tau(h^{2m-1})) \\ + \frac{\alpha}{2M} \sum_{m=0}^{2M} \mathcal{L}^v(v_{target}^m, v(h^m)) + \frac{\beta}{2M} \sum_{m=0}^{2M} \mathcal{L}^r(r_{env}^m, r(h^m)),$$

VQHybrid

$$\frac{1}{M} \sum_{m=1}^M [L^\pi(a^m, \pi(h^m)) + \alpha \mathcal{L}^v(v_{target}^m, v(h^m)) \\ + \beta \mathcal{L}^r(r_{env}^m, r(h^m))]$$

Original MuZero

Contents



- Introduction
- Model-based Planning Formulations
- VQHybrid
- Experiments

Experiments - Chess

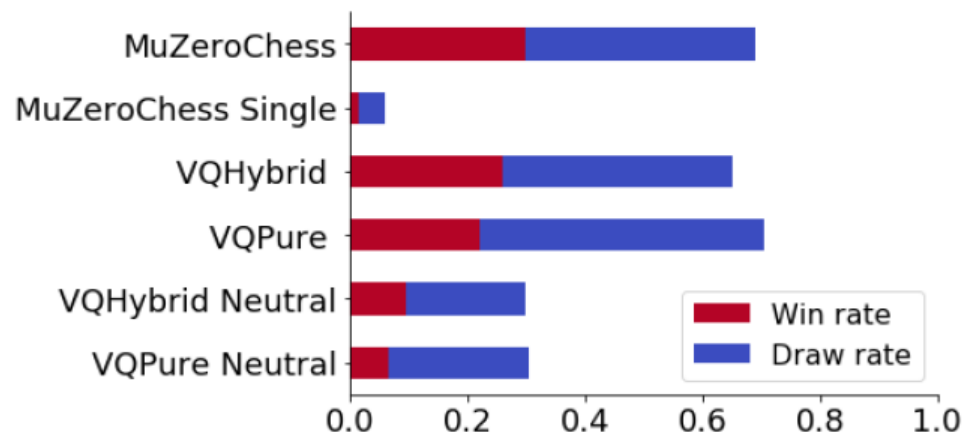


Figure 4. Performance of agents playing against Stockfish 10 skill level 15.

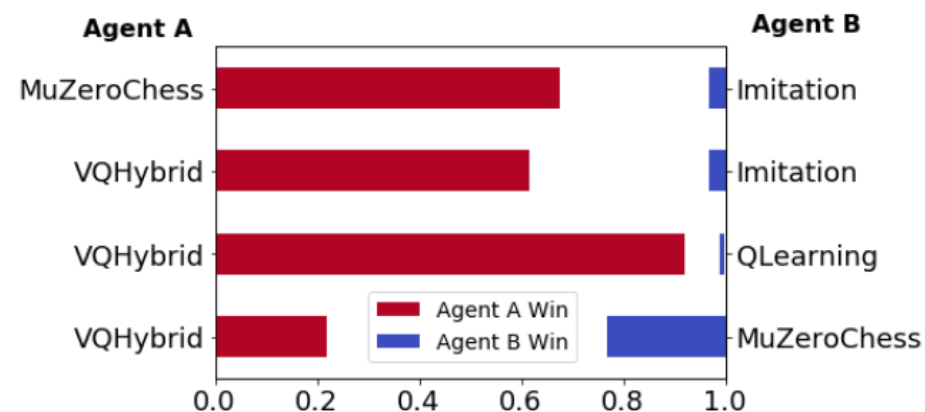


Figure 5. Performance of agents playing against each other.

$$\arg \max_k \hat{Q}(s, k) + P(k|s)U(s, k)$$

where

$$U(s, a) = \frac{\sqrt{N(s)}}{1 + N(s, a)} \left[c_1 + \log \left(\frac{N(s) + c_2 + 1}{c_2} \right) \right]$$

$$\hat{Q}(s, k) = \begin{cases} Q(s, k) & \text{cooperative} \\ 0 & \text{neutral} \\ -Q(s, k) & \text{adversarial} \end{cases}$$

Experiments – DeepMind Lab

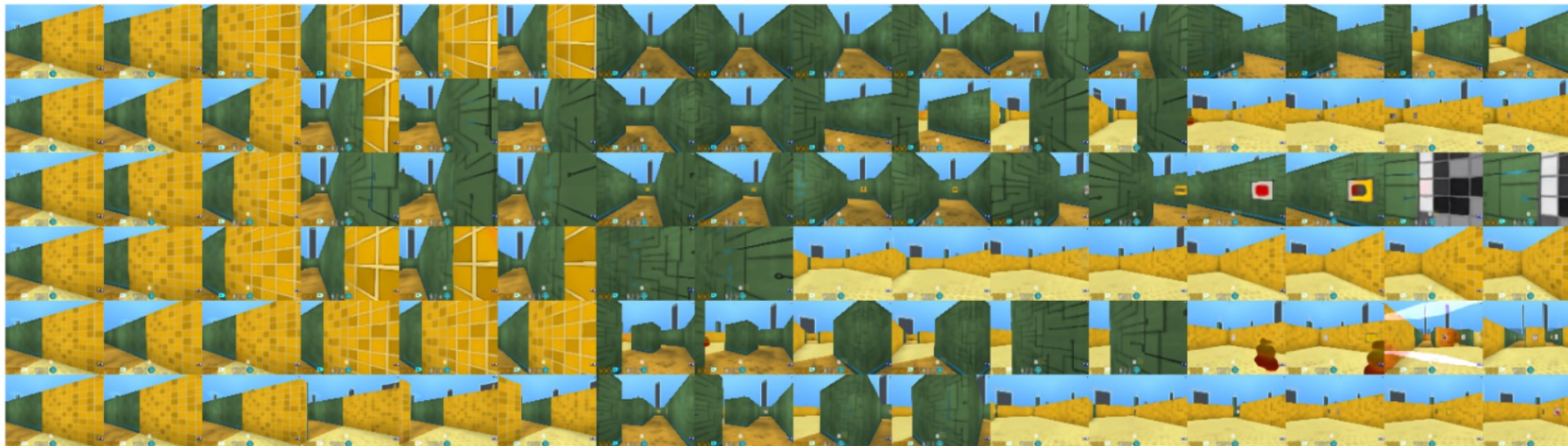


Figure 7. Different random rollouts from the same prefix video in *DeepMind Lab*. The first row is a ground-truth episode from the validation set. Each of the following rows show a sample from the model conditioned on the same 16 starting frames (not shown in the figure). Videos of more samples can be seen in <https://sites.google.com/view/vqmodels/home>.

Thanks!
